

Computer Networking: A Top-Down Approach

Chapter 2: Application Layer - Comprehensive Notes

Chapter Contents

2.1 Principles of Network Applications

2.2 The Web and HTTP

2.3 Electronic Mail in the Internet

2.4 DNS—The Internet's Directory Service

2.5 Peer-to-Peer File Distribution

2.6 Video Streaming and CDNs

2.7 Socket Programming

Download Options

Download as PDF

2.1 Principles of Network Applications

The Application Layer is where network applications (like web browsing, email, video streaming) live. It's the layer users interact with directly.

Key Concept: Applications are built by writing programs that run on **end systems** (like your laptop or a server in a data center). The network's core (routers, switches) doesn't run application code; it just forwards data. This design makes it easy to develop and deploy new applications.

2.1.1 Network Application Architectures

This refers to the high-level structure of the application. There are two main paradigms:

Client-Server Architecture

Server: An always-on, powerful host with a fixed, well-known IP address. It services requests from many clients. Examples: Web servers, email servers.

Client: Communicates with the server. It may be intermittently connected and can have dynamic IP addresses. It does not directly communicate with other clients.

Scalability: A single server can be overwhelmed. For popular services (Google, Facebook), a **data center** with massive numbers of powerful servers is used to create a virtual "super-server."

Peer-to-Peer (P2P) Architecture

Peers: There is minimal reliance on dedicated servers. Applications direct communication between intermittently connected hosts (peers), which are often desktops and laptops in homes/offices.

Self-Scalability: A key advantage. Although each peer demands resources (e.g., requests a file), each peer also *provides* resources (e.g., uploads that file to others). More users = more capacity.

Challenges: Can be less reliable and secure due to their decentralized nature. Examples: BitTorrent, blockchain applications.

2.1.2 Processes Communicating

- A **process** is a program running within an end system.
- Processes on different hosts communicate by exchanging **messages** across the network.
- In any communication session:
 - The **client** is the process that *initiates* contact.
 - The **server** is the process that *waits* to be contacted.
 - (Note: In P2P, a peer can be both a client and a server in different sessions).

The Socket Interface

- The process sends/receives messages to/from the network through its **socket**.
- **Analogy:** A process is a house, and a socket is its door. To send a message, the process shoves the message out the door. The network infrastructure delivers it to the door (socket) of the destination process.
- The socket is the **Application Programming Interface (API)** between the application and the network. The developer controls the application side of the socket but has limited control over the transport layer on the other side.

Addressing Processes

To receive a message, a process must have an address. This requires two pieces of information:

1. **IP Address:** The unique address of the host.
2. **Port Number:** A number that specifies the specific process (socket) on that host. Popular services have standard port numbers (e.g., Web/HTTP = 80, Email/SMTP = 25).

2.1.3 Transport Services Available to Applications

When choosing a transport protocol (like choosing a delivery service), an application developer cares about four key service dimensions:

Service	Description	Example Applications
Reliable Data Transfer	Guarantee that all data sent arrives correctly and completely	File transfers, email, web documents
Throughput	Rate (bits per second) of data delivery	Video streaming (bandwidth-sensitive), email (elastic)
Timing	Low delay or latency requirements	Internet telephony, video conferencing, online games
Security	Encryption, data integrity, authentication	E-commerce, online banking, private communications

2.1.4 Transport Services Provided by the Internet

The Internet offers two main transport-layer protocols to applications:

TCP Services (Transmission Control Protocol)

- **Connection-oriented service:** A "handshake" is performed to set up a connection between client and server before data flows.
- **Reliable data transfer:** Data is delivered in-order, without errors or missing pieces.
- **Congestion control:** Throttles the sender if the network is congested.
- **Does NOT provide:** Timing guarantees, minimum throughput guarantees, or built-in encryption (though this can be added with TLS).

UDP Services (User Datagram Protocol)

- **Connectionless:** No initial handshake. It's a "fire-and-forget" protocol.
- **Unreliable data transfer:** No guarantees. Data may be lost, delivered out of order, or duplicated.
- **No congestion control:** The sender can pump data into the network as fast as it wants.
- **Why use it?** It's simple, has less overhead, and is good for applications that are loss-tolerant but delay-sensitive (like Internet telephony, DNS, streaming video).

Focus on Security: TLS (Transport Layer Security)

Neither TCP nor UDP provides security (data is sent in cleartext). **TLS** is an enhancement that can be added on top of TCP to provide encryption, data integrity, and authentication. It's not a separate protocol but a layer that operates at the application layer, using its own socket API. It's the 'S' in HTTPS.

2.1.5 Application-Layer Protocols

An application-layer protocol defines the rules for communication between application processes. It specifies:

- **Types of messages** exchanged (e.g., request, response)
- **Message syntax** (the format of the messages and fields)
- **Semantics** (the meaning of the information in each field)
- **Rules** for when and how processes send and respond to messages

Examples: HTTP (Web), SMTP (email), DASH (video streaming). Protocols can be open standards (defined in RFCs) or proprietary (like Skype's).

Crucial Distinction: The *application* is the broader software (e.g., a Web browser + server + HTML standard). The *application-layer protocol* (e.g., HTTP) is just one piece of that application—it's the rulebook for communication.

2.2 The Web and HTTP

The Web is a client-server application. A user requests objects (HTML files, images, etc.) from a server, which are then displayed by the client (the browser).

2.2.1 Overview of HTTP

- The HyperText Transfer Protocol (HTTP) is the Web's application-layer protocol.
- HTTP defines the structure of messages exchanged between a Web client and a Web server.
- HTTP uses **TCP** as its underlying transport protocol.
- HTTP is **stateless**. The server maintains no information about past client requests.

2.2.2 Non-Persistent vs. Persistent Connections

How are multiple objects (e.g., a page and its images) transferred?

Non-Persistent HTTP (HTTP/1.0)

Each request/response pair is sent over a **separate TCP connection**.

Disadvantages:

1. A new TCP connection must be set up for every object, which requires time and resources (CPU, memory buffers)
2. Every object suffers a delivery delay of **at least two Round-Trip Times (RTTs)**: one to set up the TCP connection, and one to request and receive the object

Persistent HTTP (Default in HTTP/1.1+)

The server leaves the TCP connection open after sending a response. All objects for a website (or even multiple pages) can be sent over the **same single TCP connection**.

Pipelining: The client can make back-to-back requests without waiting for replies, greatly improving efficiency.

Advantages: Dramatically reduces delay, uses fewer server resources, and is much faster.

2.2.3 HTTP Message Format

There are two types of HTTP messages: **request messages** and **response messages**.

HTTP Request Message (e.g., from browser to server)

ASCII (human-readable) text.

General Structure:

1. **Request Line:** The first line. Contains:
 - **Method:** `GET` (request object), `POST` (send data to server, e.g., form input), `HEAD` (ask for headers only), `PUT` (upload file), `DELETE`
 - **URL:** Path to the requested object
 - **HTTP Version:** e.g., `HTTP/1.1`
2. **Header Lines:** Provide additional information
 - `Host: www.example.com` (mandatory in HTTP/1.1)
 - `Connection: close` (or `keep-alive`)
 - `User-agent:` (browser type)
 - `Accept-language:` (language preference)
3. **Entity Body:** Used with `POST` method to send data. Empty for `GET`

HTTP Response Message (e.g., from server to browser)

General Structure:

1. **Status Line:** The first line. Contains:
 - **HTTP Version**
 - **Status Code:** e.g., `200 OK` (success), `301 Moved Permanently` (redirect), `404 Not Found`, `500 Internal Server Error`
 - **Phrase:** Text explanation of the status code
2. **Header Lines:**
 - `Connection: close`
 - `Date:` (time response was sent)
 - `Server:` (Web server software)
 - `Last-Modified:` (crucial for caching)
 - `Content-Length:` (size of object in bytes)
 - `Content-Type:` (type of object, e.g., `text/html`)
3. **Entity Body:** The requested object itself (e.g., the HTML file, image data)

2.2.4 User-Server Interaction: Cookies

Since HTTP is stateless, **cookies** are used to allow websites to track users and maintain state.

How Cookies Work:

1. **HTTP Response:** When a user visits a site for the first time, the server's HTTP response includes a `Set-Cookie:` header with a unique ID
2. **Client-Side:** The browser stores this ID (and the server name) in a special **cookie file** on the user's machine
3. **HTTP Request:** For every subsequent request to that server, the browser automatically includes a `Cookie:` header containing the ID
4. **Server-Side:** The server uses this ID to look up the user in its backend database. It can now remember the user's preferences, shopping cart contents, login status, etc.

Components of Cookie Technology:

1. Cookie header in HTTP response
2. Cookie header in HTTP request
3. Cookie file managed by the user's browser
4. Backend database on the website's server

2.2.5 Web Caching (Proxy Servers)

A **web cache** (or **proxy server**) is a network entity that satisfies HTTP requests on behalf of an origin web server.

How it works:

1. A user's browser is configured to send all HTTP requests to the cache
2. When a request arrives:
 - If the cache has a **fresh** copy of the object, it **returns the object directly** to the client (a **cache hit**)
 - If the cache does *not* have the object, it **sends the request** to the origin server, then **stores a copy** and forwards it to the client (a **cache miss**)

Why use them?

1. **Reduce Response Time:** A cache can be geographically closer to the client than the origin server
2. **Reduce Traffic:** Reduces the amount of data flowing across an institution's access link to the Internet
3. **Help the Internet as a Whole:** By satisfying requests locally, caches reduce overall Internet traffic

The Conditional GET: Keeping Caches Fresh

Caches must ensure the objects they serve are not **stale** (i.e., outdated compared to the copy on the origin server).

How Conditional GET works:

1. When a cache requests an object from an origin server, the server includes the `Last-Modified:` header in its response
2. The cache stores the object *along with* this last-modified date
3. Later, when the cache wants to check if its object is still fresh, it can send a conditional GET request that includes the `If-Modified-Since:` header with the stored date
4. The origin server responds:
 - `304 Not Modified` with an empty body: if the object hasn't changed
 - `200 OK` with the new object: if the object *has* been modified

2.2.6 HTTP/2

HTTP/2 is a major revision designed to address performance limitations of HTTP/1.1.

Key Features of HTTP/2:

- **Multiplexing:** Multiple HTTP requests/responses can be interleaved and sent simultaneously over a single TCP

connection

- **Header Compression:** HTTP headers are compressed using the HPACK method
- **Server Push:** The server can proactively send resources to the client's cache before the client asks for them
- **Binary Framing:** HTTP/2 breaks messages into smaller binary frames instead of plain text

2.3 Electronic Mail in the Internet

Email is a classic **push** protocol. It relies on three major components:

1. **User Agents** (e.g., Outlook, Gmail app): Lets users read, reply to, forward, save, and compose messages
2. **Mail Servers:** Form the core of the email infrastructure. They run 24/7 and manage **mailboxes** and **message queues**
3. **Simple Mail Transfer Protocol (SMTP):** The application-layer protocol that moves messages from the sender's mail server to the recipient's mail server

The Journey of an Email:

1. Alice uses her **user agent** to compose an email and hits "send"
2. Alice's user agent sends the message to *her* **mail server**, where it is placed in a **message queue**
3. The client side of **SMTP**, running on Alice's mail server, opens a TCP connection to the SMTP server on Bob's mail server
4. SMTP **pushes** Alice's message over the TCP connection to Bob's mail server
5. Bob's mail server places the message in Bob's **mailbox**
6. Bob later uses his **user agent** to read the message using a **mail access protocol** to **pull** the message from the server to his local device

2.3.1 SMTP

- SMTP is the workhorse for transferring mail **between mail servers**
- Unlike HTTP, it **requires a persistent connection**
- It uses **TCP on port 25**
- It has a rigid structure with specific commands (`HELO` , `MAIL FROM` , `RCPT TO` , `DATA` , `QUIT`) and numeric reply codes
- **A key limitation:** SMTP requires that the body of all mail messages be in **simple 7-bit ASCII**

2.3.2 Mail Message Format

An email message has two main parts, separated by a blank line:

1. **Headers:** Lines in the format `Header-Name: value`
 - `From:` , `To:` , `Subject:` , `Date:` are common headers
2. **Body:** The actual content of the message
MIME (Multipurpose Internet Mail Extensions) is an extension that allows non-ASCII data (like images, audio, and application-specific files) to be sent within an email body. It adds headers like `MIME-Version:` , `Content-Type:` , and `Content-Transfer-Encoding:` to describe how the data in the body is structured and encoded.

2.3.3 Mail Access Protocols

Once an email lands in a mailbox on the server, these protocols are used to retrieve it:

POP3 (Post Office Protocol - Version 3)

A very simple protocol.

- Bob's user agent downloads messages from the server to his local machine
- Can be configured to either **delete** messages from the server after download or **keep** them
- Provides no way for Bob to create remote folders or organize mail on the server

IMAP (Internet Mail Access Protocol)

A more powerful and complex protocol.

- IMAP keeps all messages and folders (e.g., Inbox, Sent, Work) on the **server**
- The user agent acts as a window to the remote mailbox
- Allows Bob to access and manage his email from multiple different devices seamlessly

HTTP

Modern webmail services (like Gmail, Outlook.com) use HTTP for both access and sending.

- The user agent is a web browser
- The mail interface is just a website that communicates with the mail servers via HTTP APIs

2.4 DNS—The Internet's Directory Service

Hosts and routers are identified by **IP addresses** (e.g., `192.168.1.1`). The **Domain Name System (DNS)** translates human-friendly **hostnames** to machine-friendly **IP addresses**.

Other DNS Services

- **Host Aliasing:** Translating a complicated canonical hostname to a simpler alias
- **Mail Server Aliasing:** Translating an email address alias to the canonical name of a mail server
- **Load Distribution:** A single hostname can be associated with multiple IP addresses; DNS rotates the order of these addresses to distribute traffic

2.4.1 How DNS Works: A Distributed, Hierarchical Database

DNS is a brilliant example of a distributed application. It doesn't use one single server—that would be impossible to scale and maintain. Instead, it uses a vast hierarchy of servers.

There are three classes of DNS servers:

Root DNS Servers

There are 13 logical root servers (each replicated worldwide). They don't know all hostnames; they only know the locations of the **Top-Level Domain (TLD)** servers.

Top-Level Domain (TLD) Servers

These are responsible for domains like `.com`, `.org`, `.net`, `.edu`, and all country codes like `.uk`, `.jp`. They know the addresses of **authoritative DNS servers** for domains within their TLD.

Authoritative DNS Servers

Every organization with publicly accessible hosts (like web and mail servers) must provide publicly accessible DNS records that map those hosts' names to IP addresses. These records are stored on the organization's **authoritative DNS server**.

Local DNS Server

This server doesn't strictly belong to the hierarchy but is central to the process. It's typically provided by your ISP or your company/university. Your computer is configured to send all DNS queries to this **local DNS server**. It acts as a proxy, forwarding queries into the DNS hierarchy on your behalf. It also does **caching**.

2.4.2 DNS Lookup Process (Iterative vs. Recursive Query)

DNS Query Process for `www.amazon.com`:

1. Your computer sends the DNS query to its **local DNS server**
2. The local DNS server forwards the query to a **root DNS server**
3. The root server doesn't know the IP for `amazon.com`, but it knows the address of a TLD server for `.com`. It **replies with a referral** to the `.com` TLD servers
4. The local DNS server then queries one of the **.com TLD servers**
5. The TLD server doesn't know `www.amazon.com`, but it knows the address of the **authoritative DNS server** for `amazon.com`. It replies with a referral to that server
6. The local DNS server finally queries the **authoritative DNS server** for `amazon.com`
7. This server knows the IP address for `www.amazon.com` and **replies with the desired mapping**
8. The local DNS server **caches** this IP address and then sends the answer back to your computer

This process, where the local DNS server does all the work of querying different servers, is called an **iterative query**. The queries from the local DNS server to the root, TLD, and authoritative servers are iterative. The original query from your computer to the local DNS server is a **recursive query** ("please get this answer for me, no matter what it takes").

2.4.3 DNS Records and Messages

The distributed DNS database stores **resource records (RRs)**. Each RR is a 4-tuple:

(Name, Value, Type, TTL)

- **TTL** (Time to Live) determines how long the record should be cached
 - **Type** defines what **Name** and **Value** represent. The most important types are:
 - **Type A:** **Name** is a hostname, **Value** is its **IPv4 address**
 - **Type AAAA:** For **IPv6 addresses**
 - **Type NS:** **Name** is a domain, **Value** is the hostname of the **authoritative DNS server** for that domain
 - **Type CNAME:** **Name** is an alias hostname, **Value** is the **canonical (real) hostname**
 - **Type MX:** **Name** is a domain, **Value** is the name of a **mail server** associated with that name
- DNS Messages** have a header and sections for questions, answers, authority records, and additional records. They are typically carried within **UDP** datagrams to minimize overhead, using port 53.

2.5 Peer-to-Peer File Distribution

This section contrasts the client-server and P2P architectures for a specific task: distributing a large file from one server to a large number of clients (peers).

Client-Server Architecture

The server must send a *separate copy* of the file to each of the N peers. This places an enormous upload burden on the server. The **distribution time** (time to get the file to all peers) increases linearly with the number of peers (N). It does **not scale well**.

P2P Architecture

Peers are not just consumers; they are **resources** that help distribute the file. After a peer downloads a portion of the file, it can begin uploading that portion to other peers.

- The server's upload capacity is shared
- The total **upload capacity of the entire system** increases as more peers join
- As a result, the distribution time for a P2P system can be *much* lower than for a client-server system, especially for a large number of peers (N). It **scales well**

The textbook uses a simple model to show that the minimum distribution time for P2P systems increases in proportion to the logarithm of N (or similar), which is dramatically better than the linear increase of the client-server model.

BitTorrent is the most famous modern P2P file-sharing protocol, using techniques like splitting files into **chunks** and having peers trade ("tit-for-tat") chunks they have for chunks they need.

2.6 Video Streaming and Content Distribution Networks

Video is the dominant consumer of Internet bandwidth today. Streaming has unique challenges: it requires **high throughput** and is highly **delay-sensitive**.

2.6.1 Internet Video

- Videos are **encoded** at a specific bitrate (e.g., standard definition, HD, 4K)
- The **throughput** of the network path from server to client must be *at least* as large as the video's bitrate for continuous playback
- **Key Challenge:** Internet throughput can be highly variable due to congestion

2.6.2 HTTP Streaming and DASH

Early video streaming used proprietary protocols over UDP. Today, **almost all streaming is done over HTTP**.

How HTTP Streaming Works:

1. The video file is **prepared** on the server: it is **encoded** at multiple different quality levels and **divided** into small **chunks** (2-10 second segments)
2. A **manifest file** is provided to the client, which contains the URLs for the different versions of each video chunk
3. The client application **dynamically** selects which chunk to request next based on its current estimate of available bandwidth
 - If bandwidth is high, it requests a high-bitrate chunk
 - If bandwidth drops, it requests a lower-bitrate chunk

This technique is called **DASH (Dynamic Adaptive Streaming over HTTP)**.

Benefits:

1. The client adapts to network conditions, minimizing rebuffering
2. Using HTTP means the video traffic looks like normal web traffic
3. The server is greatly simplified; it just stores and serves files

2.6.3 Content Distribution Networks (CDNs)

A single mega-video server would be overwhelmed by global demand. The solution is to **distribute** the content.

A **CDN** is a network of geographically distributed servers, designed to deliver content efficiently to users.

How CDNs Work:

1. The CDN **stores copies** of content (like popular videos) in its servers around the world
2. The core idea is to **push content closer to the user**
3. When a user requests a video, the CDN's system **redirects** the request to a CDN server that is close to the user
4. This server then delivers the content

How does a user get redirected to a CDN server?

The secret is in **DNS**. When your player requests a video, it asks for a hostname owned by the CDN. Your DNS query gets redirected to the CDN's DNS server, which returns the IP address of the **best** CDN server for you based on your location.

2.7 Socket Programming: Creating Network Applications

This section provides a hands-on introduction to network application development using the **socket API**.

Key Concepts:

- The **socket** is the door between the application process and the transport-layer protocol (TCP or UDP)
- The programmer must choose between a **TCP socket** (reliable, connection-oriented stream) or a **UDP socket** (unreliable, connectionless datagram)
- The process is different for the client and server sides

2.7.1 Socket Programming with UDP

UDP is connectionless. There is no initial handshake. The sender explicitly attaches the destination address to each packet.

UDP Client Steps:

1. Create a socket
2. Create a datagram packet with the server's IP address and port number
3. Send the datagram packet into the socket
4. (Optional) Wait to receive a reply packet from the server

UDP Server Steps:

1. Create a socket and **bind** it to a specific port number
2. Wait to receive a datagram packet from a client
3. Process the received datagram
4. Create a reply datagram packet, attaching the client's address
5. Send the reply packet back to the client

2.7.2 Socket Programming with TCP

TCP is connection-oriented. The client and server must first establish a connection before exchanging data.

TCP Client Steps:

1. Create a socket
2. **Initiate a connection** to the server at its IP address and port number
3. Once the connection is established, **send** data into the socket
4. **Read** data sent back from the server from the socket
5. **Close** the connection

TCP Server Steps:

1. Create a **welcoming socket** and **bind** it to a specific port number
2. **Wait** on the welcoming socket for a connection request from a client
3. When a request arrives, create a **new socket** dedicated to that client
4. **Read** data from the new (connection) socket
5. **Send** data to the client through the same connection socket
6. When done, **close** the connection socket

The chapter concludes with code examples that implement a simple client-server application, such as a client sending a string to a server, which converts it to uppercase and sends it back.

Computer Networking: A Top-Down Approach - Chapter 2 Notes
Based on the textbook by James F. Kurose and Keith W. Ross